Minicurso Docker



Aula 2

COMP PET

UFU Universidade Federal de Uberlândia

facom

Ambiente

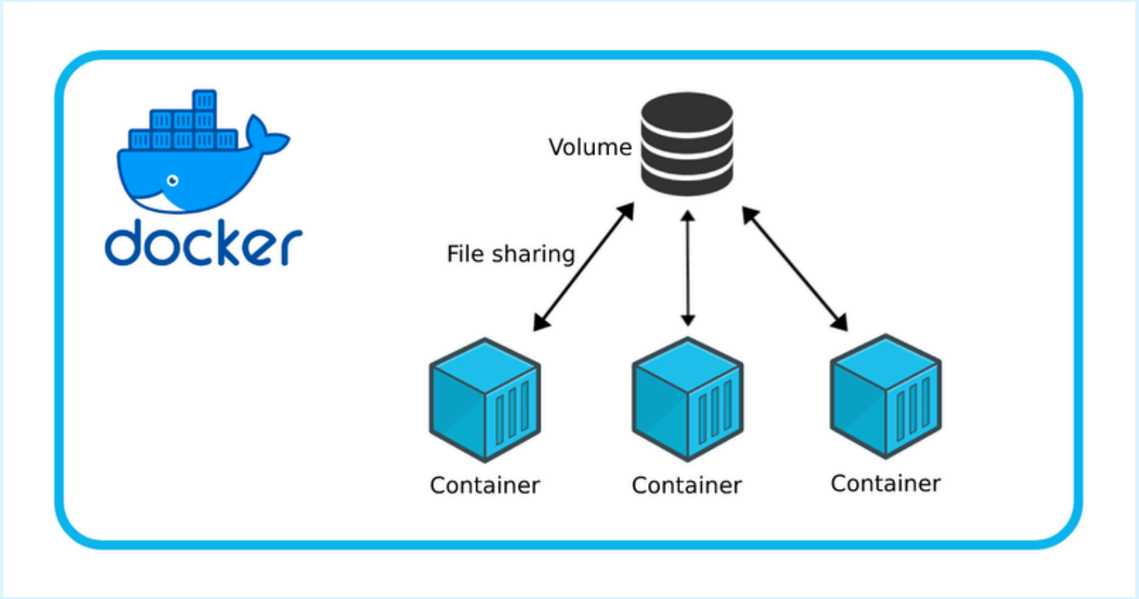
- O Docker é usado principalmente por meio de comandos no terminal
- Vamos ver alguns dos mais importantes
- Para testar sem precisar baixar:
- https://labs.iximiuz.com/playgrounds/docker
- https://killercoda.com/docker

Persistência de dados?

- Como vimos, os dados em contêiners não são persistentes
- Para garantir persistência, utilizaremos *bindmounts* e *volumes*
- Já vimos *bindmounts*: é a opção *-v* da aula passada.
- Mas nem sempre são a melhor opção...

Volumes?

- Para garantir a segurança e performance, o recomendado é utilizar volumes
- Os volumes persistem na máquina hospedeira e são gerenciados pelo próprio docker
- Vários contêiners podem acessar o mesmo volume



Tipos

- 4 formas de montar diretórios em um contêiner:
  - bind: mapeia diretório
  - volume: mapeia um volume
  - tmpfs: faz um mapeamento temporário (RAM)
  - image: permite montar uma imagem dentro de outra (apenas leitura)

Como iniciar volumes

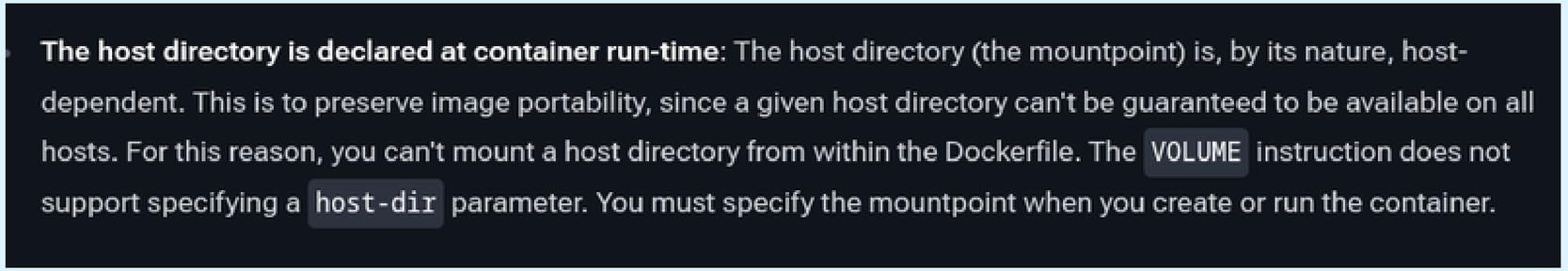
- Volumes podem ser criados automáticamente durante a inicialização do contêiner:
  - *-v nome-vol:dir_no_conteiner*
  - *--mount type=tipo,src=host,dst=cont*
- Volumes do tipo *bind* ocorrem por meio da rede (pode ser lento!)
- Atenção: pontos de montagem sobreiam arquivos locais dos contêiners

Comandos importantes

- Com *docker volume*:
  - *create:* cria volumes no sistema
  - *ls*: lista volumes no sistema
  - *inspect:* mostra detalhes sobre algum volume
- Quando uma imagem requer um contâiner, esse é criado automáticamente pelo Docker

No dockerfile

- Volumes também podem ser criados em um dockerfile
  - VOLUME ‘volume’
- Entretanto, isso não é recomendado! Pois não é possível específicar um diretório no Dockerfile



Vários contêiners?

- Suponha que tenhamos uma aplicação grande, com vários componentes
- Idealmente cada componente deve estar separado em um contêiner diferentes
- Como podemos iniciar todos os contêiners fácilmente?
- A solução pra isso é o Docker compose!

Docker compose?

- O docker compose permite definir como iniciar vários containers de uma só vez
- Um arquivo *compose.yaml* indica a ordem e como cada contêiner deve ser iniciado
- comandos principais *docker compose*:
  - up: lê o arquivo e inicia os contêiners
  - down: para os contêiners e os remove
  - ps: lista serviços ativos

Docker compose?

- O docker compose permite definir como iniciar vários containers de uma só vez
- Um arquivo *compose.yaml* indica a ordem e como cada contêiner deve ser iniciado
- comandos principais *docker compose*:
  - up: lê o arquivo e inicia os contêiners
  - down: para os contêiners e os remove
  - ps: lista serviços ativos

O arquivo

- O *compose.yaml* tem a seguinte estrutura:
  - name: *nome_app*
  - services
    - *nome_servico:*
      - *configs_serviço*
  - volumes:
    - nome-volume
      - *configs_volume*

O arquivo

- O *compose.yaml* tem a seguinte estrutura:
  - networks
  - configs:
  - secrets:
- Essas três estruturas são opcionais, mas permitem maior controle sobre as aplicações

Atributos de *service*

- Como vimos, os serviços podem receber atributos que definem suas configurações
  - *build:* define o caminho de criação da imagem
  - *image:* define a imagem daquele serviço
  - *ports:* permite publicar portas (similar ao *-p*)
  - *environment:* permite definir variáveis de ambiente (similar ao *-e*)

Atributos de *service*

- Continuando...
  - *depends_on*: indica que esse serviço depende de outro
    - *condition*: permite configurar o *depends_on*
  - *develop:* especifica como um contêiner deve se manter sincronizado (para desenvolvimento)
    - *watch*: regras que devem ser seguidas

.dockerignore

- Um arquivo *.dockerignore* também pode ser adicionado para que o Docker não publique esses arquivos
- Similar ao *.gitignore*
- Usado para segurança ou economizar espaço, nem todos os arquivos precisam/podem ir ao público

Outros exemplos

- Para quem tiver curiosidade, o Docker mantém um repositório com diversos exemplos de *compose*:
- https://github.com/docker/awesome-compose

Como se comunicam?

- Por trás dos panos, os contêiners estão isolados, então como funciona a comunicação entre eles?
- Por meio da rede! O docker compose automáticamente configura uma rede interna e expões as portas necessárias para que os contêiners comuniquem entre si.
- Podemos ver a rede com *docker network ls*
- Também podemos criar nossa própria rede com *docker network create*

Redes

- Por padrão, todos os contêiners dentro de uma rede tem acesso à outros naquela mesma rede
- Para contêiners em diferentes redes, o acesso ocorre como se fosse a distância (necessário publicar portas)
- Em redes criadas por usuários, o acesso pode ser feito pelo nome do contêiner (DNS automático)
- Os conceitos aprendidos em sistemas distribuidos se aplicam aqui, mas fogem do escopo desse curso.

Muito obrigado!



Dúvidas?

COMP PET

UFU Universidade Federal de Uberlândia

facom